“SigniVerse: An Educational Game for learning Jordanian Sign Language (LIU) through Gesture Recognition”

Abstract

The Deaf and Hard-of-Hearing community in Jordan lacks self-paced, interactive resources for learning Jordanian Sign Language (LIU). This paper introduces "SigniVerse", a serious educational game that supports children who are learning Jordanian Sign Language by providing real-time feedback through sign recognition. The game teaches foundational categories such as the alphabet, numbers, and everyday household items. Each category is taught through a minigame along with its respective sign recognition model.

The use of sign recognition models enabled immediate feedback, ensuring that the player learns and performs the sign correctly.

Exploratory interviews were conducted with individuals from the DHH community. The interviews emphasized the importance of early-age learning and informed the signs selection.

The system is designed to be scalable, it can accommodate the addition of new vocabulary and minigames. Furthermore, the custom dataset will be made publicly available to support future research.

This study evaluates the usability and technical performance of the game rather than its educational outcomes, which are reserved for future work.

Chapter 1: Introduction

Overview

This paper intends to design an interactive game that is aimed at teaching sign language to Deaf and Hard of Hearing (DHH) children. The game uses a technology that employs computer vision in recognizing the child's hand gesture and offering real-time feedback. In doing so, the learning process becomes more efficient since the child learns sign language in a self-dependent manner.

The game consists of several tutorials, animated videos, and various other games, all of which aim to make learning fun for the children. This paper intends to propose an innovative approach towards inclusive education.

Problem Statement

The learning environment for sign language LIU at present is limited only to certain centers. Moreover, other available digital learning materials include videos, which requires one to be able to read and listen in order to understand them. Thus, DHH children cannot learn independently outside classrooms.

Project Aim and Objectives

Project aim:

The aim of this project is to develop an interactive educational game designed to support deaf and hard-of-hearing (DHH) children who are learning Jordanian sign language in an engaging, accessible way.

Project Objectives:

- To design and develop an interactive serious game that supports Jordanian Sign Language (LIU) practice for children aged 6-10 through engaging minigames and a reward system.
- To collect and publish a labelled dataset of LIU signs for model training and supporting LIU research.
- To design a scalable game that supports the addition of new vocabulary and categories.
- To evaluate the game through user-based testing focusing on UI quality, usability, and user satisfaction.

Chapter 2: Background and Related Work

The development of deep learning models gave birth to many applications for recognizing gestures for sign languages. Rastgoo et al. (2021) give a review of the current state of affairs, showing that the use of LSTM recurrent neural networks in video processing, coupled with frameworks like Faster R-CNN, YOLO, and SSD for spatial features' extraction, shows the best results in this field. This means that the use of LSTM networks, which are one of RNN's subtypes, is reasonable. In the literature, there are already several proven cases, such as SIGNIFY by Ulrich et al. (2024) and PopSignAI by Starner et al. (2021).

Despite the wide variety of educational applications for sign language, there is still a deficiency of solutions that employ computer vision techniques to make the process of learning more engaging. SIGNIFY uses games for practicing the manual alphabet but does not go beyond the scope of one or two actions. The most similar solution, PopSignAI, implements real-time gesture recognition technology for making the educational process easier and faster.

SigniVerse stands out among available solutions due to the following aspects:

1. Linguistic Localization: Unlike other prototypes, SigniVerse is focused only on Jordanian Sign Language (LIU), which makes it a great local resource.
2. Diversity of Gameplay: SigniVerse includes a set of different types of minigames compared to other solutions.

Chapter 3: Design Details

The chapter outlines the methodological framework of the system. It is divided into three main components. (1) System architecture, (2) Data acquisition, (3) Game development & Integration.

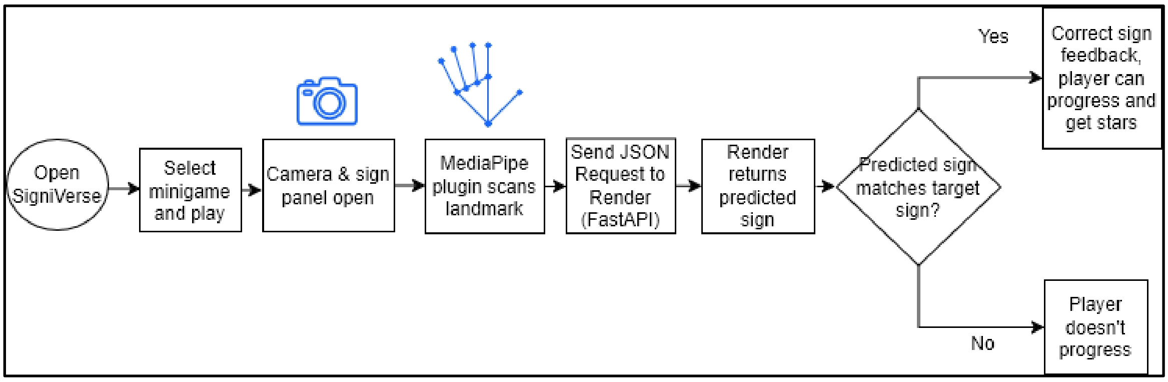
3.1 System Architecture

SigniVerse is a platform that integrates Unity and computer vision seamlessly through the use of plugins and a backend server.

The system workflow consists of choosing a minigame to play, having a “sign language panel” open throughout the game to teach the sign and have the player perform it. The sign language panel consists of an avatar performing the sign, a speech bubble containing the written sign, and a camera feed which allows the user to mimic the avatar and get feedback when performing the sign.

The feedback mechanism works by leveraging the Unity MediaPipe Plugin to take the hand landmarks of the user, then send the landmarks to the backend, consisting of python code hosted on Render. The inference is conducted in the backend and the prediction is returned.

Validation is conducted on Unity, and the appropriate feedback is displayed when the user performs the sign correctly.



3.2 Data Acquisition

The dataset is curated through a controlled recording process involving four signers. For each sign, 100 videos were recorded, with each video spanning 3 seconds at a frame rate of 30 frames per second (fps). The MediaPipe Hand Landmarker framework is employed to capture the coordinates of 21 distinct hand landmarks. Rather than training on raw image data, each model is trained on landmarks to reduce chances of overfitting. A Long Short-Term Memory (LSTM) Network, a type of Recurrent Neural Networks (RNN) was used to train the models. A modular approach of training a separate model for each set of signs was taken to improve classification accuracy while minimizing inference latency.

Dataset Size

- Alphabet Signs: 35 classes, approximately 3500 recorded sequences and 7000 sequences after augmentation.
- Number Signs: 10 classes, approximately 2000 sequences after augmentation.
- Home Signs: 22 classes, approximately 2200 sequences.

- Sequence length: 90 frames per sequence.
- Total classes: 67 classes.
- Total sequences: approximately 11,200 sequences.

Labeling Method

Each sign class was assigned a unique integer label according to its position in the predefined actions list. Every sequence was mapped to its corresponding label and stored for supervised learning.

Preprocessing Steps

- Recording videos using webcams.
- Applying image enhancement techniques such as CLAHE and sharpening filters.
- Extracting hand landmarks using MediaPipe Hands.
- Using one hand for Alphabet and Number datasets and both hands for Home Signs.
- Extracting landmark coordinates (x, y) for Alphabet and Number signs and (x, y, z) world landmarks for Home Signs, as the signs required two hands.
- Applying horizontal flipping augmentation for Alphabet and Number datasets to support both left-hand and right-hand recognition.
- Saving extracted landmarks as NumPy (.npy) files.
- Organizing samples into sequences of 90 consecutive frames.
- Converting sequences into NumPy arrays suitable for LSTM training.
- Validating frame existence and feature consistency before training.

Dataset Split

The datasets were divided into training, validation, and testing sets using StratifiedGroupKFold to maintain balanced class distributions, prevent data leakage, and improve model generalization.

3.3 Game Development & Integration

The game environment was developed using Unity 6, using an iterative agile methodology. Each minigame was developed in increments aligned with the availability of each model. To incorporate computer vision and immediate feedback, a MediaPipe Unity Plugin was used. The minigames are represented as planets, leaving the development team with the ability to add more minigames in the future.

Chapter 4: Implementation Details

In this section, there is a brief overview of software tools utilized for building a SigniVerse Application.

- **Game Engine**: Unity (version 6000.3.6f1 LTS) is employed as the primary game engine due to numerous opportunities for creating 3D and 2D game environment as well as third-party AI plugins installation.
- **Programming Language**: the chosen programming language for the game implementation code is C#.
- Third-party AI/Computer Vision Related Tools:
  - MediaPipe from Google - for detecting hand landmarks with extraction of 21 points in 3D space from the camera stream.
  - LSTM - (Long Short-Term Memory) – specialized recurrent neural network that recognizes and classifies movement sequences of hand landmarks in order to identify and classify dynamic sequential gestures.
  - TensorFlow/TensorFlow Lite – for training LSTM.

- **Backend API Framework**: FastAPI is the selected framework that can build up an API endpoint and make a link between Unity client and ML model. This API receives hand landmark coordinates delivered by Unity client via HTTP POST request, recognizes and classifies gestures and sends the classification result to the client application.
- **Hosting Platform**: Render, which operates as a unified cloud computing platform for hosting and deploying FastAPI server side code. It is worth mentioning that it allows for automatic deployment of the source code stored in GitHub repository and generation of a stable production URL, which will be utilized by Unity client in terms of interaction with the backend ML model.
- **Avatar Animation Platform**: Kling AI was selected for designing instructional sign language content for the game. In particular, this tool is utilized for animations of an astronaut performing sign language gestures.
- **Integrated Development Environment (IDEs):** Visual Studio for the implementation of the C# script.
- **Source Code Version Control**: GitHub/GitHub Desktop.

Local Data (PlayerPrefs)

PlayerPrefs in Unity were selected as a straightforward data persistence framework that supports saving data about the progress made by a child in terms of learning sign language:

- Progress Tracking: Stores data about unlocked level, score and planets completed flag;
- Logic: Uses key-value pair approach for ensuring data saving/Loading once exited the app.

Chapter 6: Experiments and Results

6.1 Introduction

This chapter presents the testing and installation process of the game. The purpose of testing is to ensure that the game works correctly, is stable, easy to use, and provides a good user experience. The chapter includes functional testing, usability testing, cooperative evaluation, installation steps, and a user manual that explains how to play the game.

6.2 Testing Environment

The game was tested on Unity Editor and on the target device to make sure that all gameplay features work correctly. The testing focused on player movement, jumping, collecting stars, losing, winning, level unlocking, and navigation between game screens.

The game was tested on Unity Editor and on the target device to make sure that all gameplay features work correctly. The testing focused on player movement, jumping, collecting stars, losing, winning, level unlocking, and navigation between game screens.

Table 1:Testing Environment

Item	Description
Game Engine	Unity
Programming Language	C#
Testing Device	Laptop / Android Phone
Operating System	Windows / Android
Input Method	Keyboard / Touch / Motion detection
Build Type	APK / Windows Build / Unity Editor
Testers	Students / Friends / Family members

6.3 Functional Testing

Table 2:Functional Testing

Test Case ID	Feature	Test Steps	Expected Result	Actual Result	Status

TC1	Start Game	Open the game and press the button Start	The planets page opens	Planets page opened	Pass
TC2	Numbers mini game	Press on the numbers planet in planets page	The numbers game opens	Numbers game opened	Pass
TC3	Joystick	Move it in different directions	The character moves in the same directions	The character moved in the same directions	Pass
TC4	Collect Star	Touch/collide with a star	Star count increases and the panel opens to do the sign	Star count increased and the panel opened to do the sign	Pass
TC5	Win panel	Collect all 10 stars	Win panel opens	Win panel opend	Pass
TC6	Alphabets mini game	Press on the alphabets planet in planets page	The alphabets game opens	The alphabets game opened	Pass
TC7	Player Movement	Use the control keys/buttons	The player moves correctly	Player moved correctly	Pass
TC8	Jump	Press jump button	The player jumps once when on ground	Jump worked	Pass
TC9	Collect Stars	Touch/collide with a star	Star count increases	Star count increased	Pass
TC10	Obstacle	Hit an obstacle	Lose panel	Lose panel	Pass

	Collision		appears	appeared	
TC11	Restart Level	Press Restart after losing	Same level restarts	Level restarted	Pass
TC12	Win Level	Reach finish line	Win panel appears	Win panel appeared	Pass
TC13	Next Level	Press next level	Next level opens	Next level opened	Pass
TC14	Level Unlocking	Win Level 1	Level 2 becomes unlocked	Level 2 unlocked	Pass
TC15	Back button	Press back button	Player returns to main menu	Main menu opened	Pass
TC16	Home mini game	Press on the home planet in planets page	The home game opens	The home game opened	Pass
TC17	Open bedroom	Press on bedroom	The room opens	The room opened	Pass
TC18	Open bathroom	Press on bathroom	The room opens	The room opened	Pass
TC19	Open living room	Press on living room	The room opens	The room opened	Pass
TC20	Open kitchen	Press on kitchen	The room opens	The room opened	Pass
TC21	Rooms items	Pess any item	A puzzle for this item will open	The puzzle opened	Pass
TC22	Market button	Press market button on planets page	The market opens	The market opened	Pass
TC23	Buy character	Press buy on market	The character purchase	The character purchased	Pass

6.4 Heuristic Evaluation

Heuristic evaluation was conducted to check the usability of the game interface. The evaluation focused on whether the game screens, buttons, feedback messages, and navigation are clear and easy to use.

Table 3:Heuristic Evaluation

No.	Heuristic	Description for the Game
H1	Visibility of System Status	The game should show score, stars, level status, win/lose feedback clearly.
H2	Match Between Game and Real World	Icons, buttons, and instructions should be familiar and easy to understand.
H3	User Control and Freedom	The player should be able to restart, or return to menu easily.
H4	Consistency and Standards	Buttons, colors, fonts, and panels should be consistent across all levels.
H5	Error Recovery	The game should show clear feedback when the player loses or cannot continue.
H6	Error Prevention	The game should prevent unfair losing, repeated jumps, or broken level transitions.
H7	Recognition Rather Than Recall	Controls and objectives should be visible instead of making the player guess.
H8	Flexibility and Efficiency	Players should be able to restart or move to the next level quickly.
H9	Aesthetic and Minimalist Design	The game UI should be simple and not crowded.
H10	Help and Documentation	The game should provide instructions or a short tutorial.

Summary table

Table 4:Summary table

Heuristic	**Issues Found**	**Severity**
H1	1	Minor
H4	1	Minor
H5	1	Minor
H7	2	Minor
H10	1	Minor

6.5 Cooperative Evaluation

Cooperative evaluation was performed with three participants. Each participant was asked to play the game and complete a set of tasks while speaking about any difficulties they faced. The goal was to identify usability problems and measure how easy the game is to understand.

Table 5:Cooperative Evaluation

Criteria	**Participant 1**	**Participant 2**	**Participant** 3	**Participant** 4
Gender	Female	Male	Female	Male
Age	9 years old	8 years old	18 years old	22 years old
Educational Level	3rd grade	2nd grade	University Student	University Student
Gaming Experience	Medium	Beginner	Medium	Medium

Tasks

Table 6

1	Start the game
2	Understand the controls

3	Collect a star
4	Avoid obstacle
5	Restart after losing
6	Win the level
7	Go to next level
8	Return to main menu
9	Open the market
10	Buy character

6.6 Post-Test Questionnaire

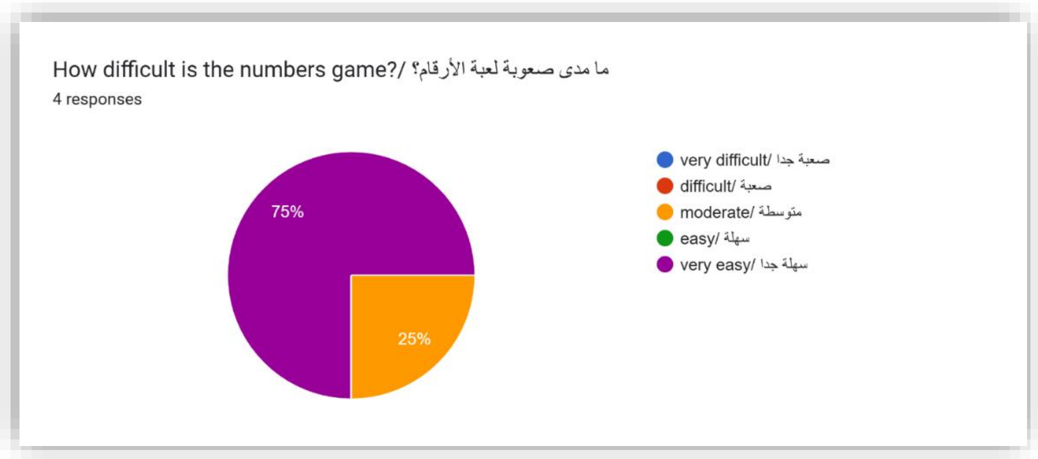


Figure 1:Post-Test Questionnaire

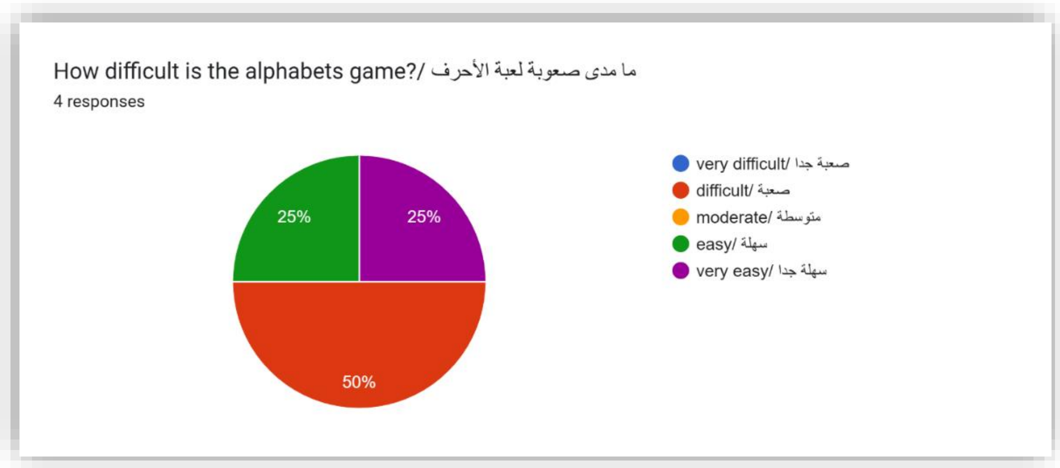


Figure 2:Post-Test Questionnaire

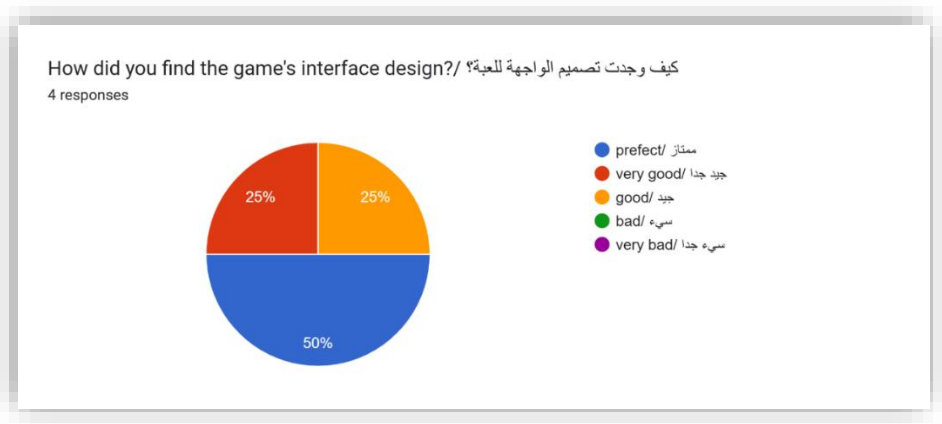


Figure 3:Post-Test Questionnaire

Figure 4:Post-Test Questionnaire

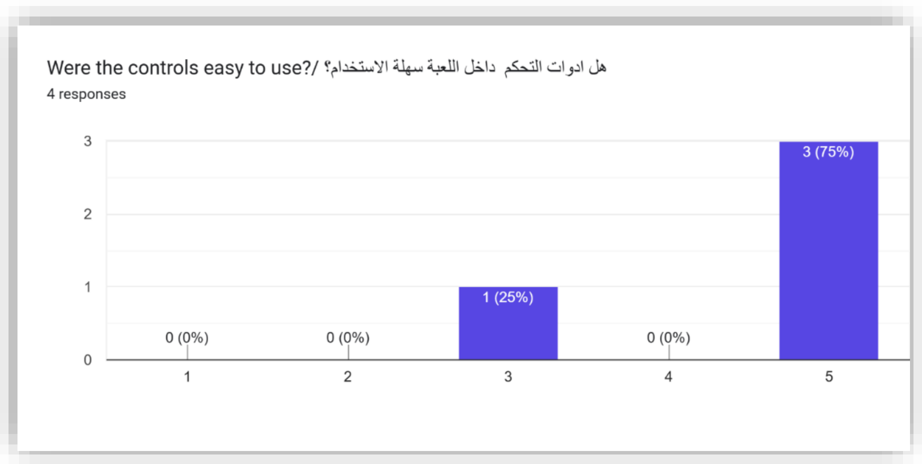


Figure 5:Post-Test Questionnaire

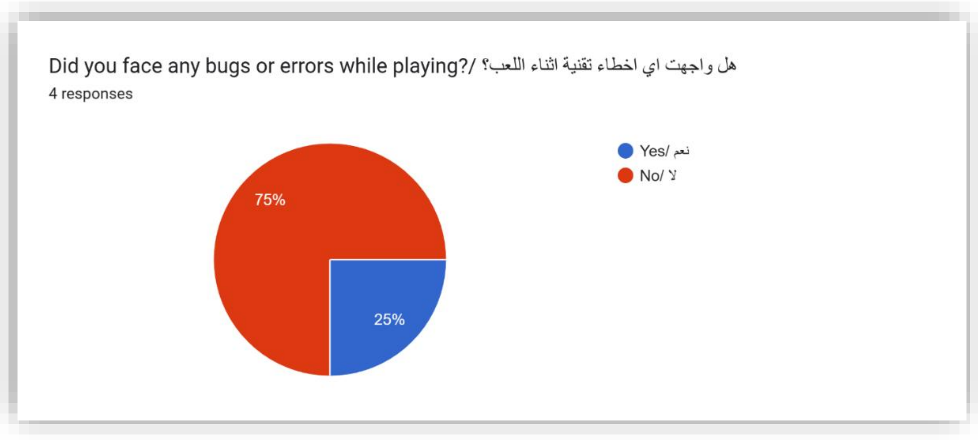


Figure 6:Post-Test Questionnaire

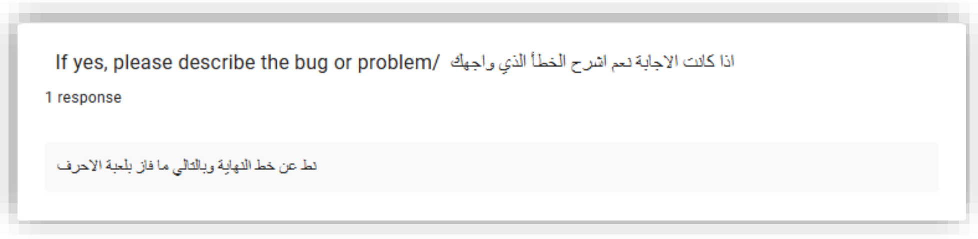

If yes, please describe the bug or problem/ اذا كانت الاجابة نعم اشرح الخطأ الذي واجهك
1 response
نط عن خط النهاية وبالتالي ما فاز بلعبة الاحرف

Figure 7:Post-Test Questionnaire

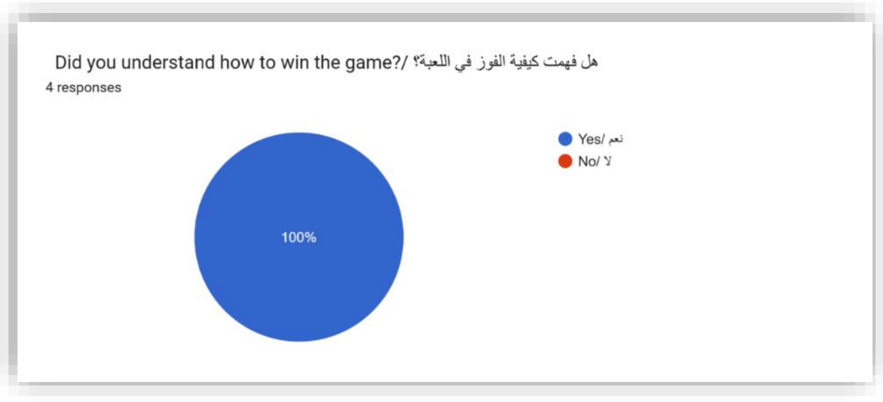


Figure 8:Post-Test Questionnaire

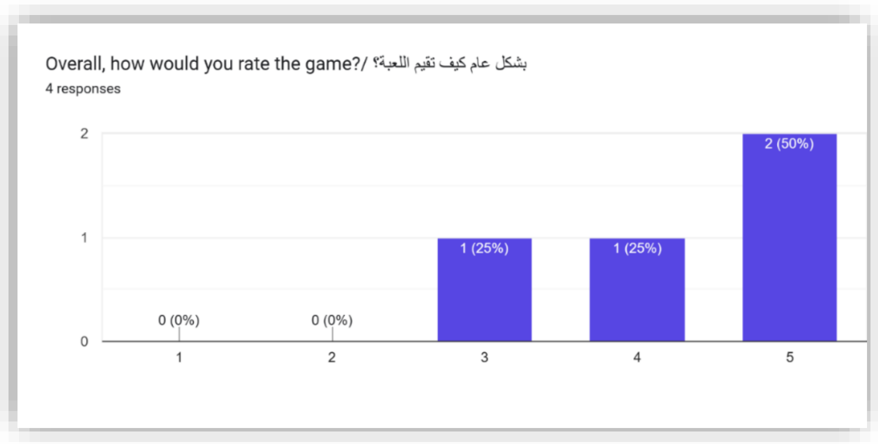


Figure 9:Post-Test Questionnaire

Figure 10:Post-Test Questionnaire

Chapter 7: Conclusion and Future Work

7.1 Conclusions to the Project

In order to provide support for educating deaf children on their sign language, the primary aim of the SigniVerse Project was to create an experience that combines artificial intelligence with an immersive game-like atmosphere. It is apparent that a pipeline from Unity software to a temporal deep learning model (MediaPipe and LSTM) has been successfully designed. This proves that computer vision systems may be used interactively without special hardware.

7.2 Project Strengths and Weaknesses

7.2.1 Strengths

- Learning through Gamification

The gamification process involved in the SigniVerse, as well as its reward system, makes learning an enjoyable activity.

- Universal Adaptability

Although the SigniVerse is specially designed for hearing-impaired children, it is also equally appropriate for non-hearing-impaired children because it promotes knowledge about Jordanian Sign Language.

- Free from Cost

Accessing the SigniVerse does not involve any particular hardware requirement. All a person needs to access this program is a webcam on his/her computer.

- Affordability and Availability

Being an internet-based program for free, the SigniVerse allows children to learn about Jordanian Sign Language without spending money.

7.2.2 Weaknesses

- Light Sensitivity:

Inadequate lighting could lead to inaccurate identification of hand landmarks using MediaPipe due to insufficient light.

- Local Progress Tracking:

The progress made by the user is stored locally on the device and not accessible by other devices.

7.3 Future Work

The further development of the SigniVerse application is recommended in order to transform it into a commercialized educational application. Such improvements include:

- Expansion of the Signatures Vocabulary

Training LSTM on various combinations of hand gestures up to making complete sentences.

- Mobile and Cross-Platform Deployment

Reference list

Rastgoo, R., Kiani, K., & Escalera, S. (2021). *Sign language recognition: A deep survey.* Expert Systems with Applications, 164, 113794. https://doi.org/10.1016/j.eswa.2020.113794

Ulrich, L., Carmassi, M., Sartini, F., Natarelli, F., & Moos, S. (2024). *SIGNIFY: Leveraging machine learning and gesture recognition for sign language teaching through a serious game.* Applied Sciences, 14(1), 336. https://doi.org/10.3390/app14010336

Starner, T., et al. (2021). *PopSignAI: Making American Sign Language learning fun with games and computer vision.* In Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems. https://doi.org/10.1145/3411764.3445263